

Lab 5: Value-Based Reinforcement Learning

StudentID:E94111253

Date:2026/05/05

1. Introduction

This lab implements value-based reinforcement learning using Deep Q-Networks (DQN). Task 1 applies vanilla DQN to the low-dimensional CartPole-v1 environment, while Tasks 2 and 3 extend DQN to the visual Pong-v5 environment. Task 3 further incorporates Double DQN, Prioritized Experience Replay (PER), and 3-step return to improve learning efficiency and stability.

Table1. Summary of implemented components in each task.

	Task1	Task2	Task3
Environment	CartPole-v1	Pong-v5	Pong-v5
Input	4-D state	4 stacked frames	4 stacked frames
Network	MLP	CNN	CNN
Replay	Uniform	Uniform	PER
Target	DQN	DQN	Double DQN
Return	1-step	1-step	3-step

2. Implementation

2.1 Bellman Error

For vanilla DQN, given a transition $(s_t, a_t, r_t, s_{t+1}, d_t)$, the target value is computed as:

$$y_t^{DQN} = r_t + \gamma(1 - d_t) \max_{a'} Q_{target}(s_{t+1}, a')$$

where d_t is the terminal flag. The DQN objective minimizes the difference between the predicted Q-value and the target value:

$$L(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (y_i - Q_{online}(s_i, a_i))^2$$

In our implementation, the online Q-value is obtained by selecting the Q-value corresponding to the executed action, while the target value is computed by the target network. Task 1 uses MSE loss, while Tasks 2 and 3 use Huber loss. In Task 3, the Huber loss is further weighted by PER importance-sampling weights. In the code, the predicted Q-value is computed by `q_net(states).gather(1, actions)`, while the target value is computed using

`target_net(next_states).max(1)[0]`. The done flag is used to mask the bootstrap term so that terminal states do not include future Q-values.

2.2 Double DQN

Vanilla DQN may overestimate Q-values because the same target network is used for both action selection and evaluation. Double DQN separates these two roles:

$$y_t^{DDQN} = r_t + \gamma(1 - d_t)Q_{target}(s_{t+1}, \arg \max_{a'} Q_{online}(s_{t+1}, a'))$$

In our implementation, `self.q_net` is used to compute `next_actions`, and `self.target_net` is used to evaluate the Q-values of those selected actions. This Double DQN update is used only in Task 3 as one of the DQN enhancements. Task 1 and Task 2 use the vanilla DQN target instead. In Task 3, `q_net` selects the next action, while `target_net` evaluates the selected action..

2.3 Prioritized Experience Replay (PER)

Prioritized Experience Replay samples important transitions more frequently. Each transition i is assigned a priority based on the magnitude of its TD error:

$$p_i = |\delta_i| + \epsilon$$

where ϵ is a small constant to ensure that every transition has non-zero probability.

The sampling probability of transition i is:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

where α controls the strength of prioritization.

In our implementation, the replay buffer stores both transitions and their priorities. During sampling, transitions are drawn according to $P(i)$, and the corresponding normalized importance-sampling weights are applied to the element-wise Huber loss. PER is used only in Task 3. Task 1 and Task 2 use uniform experience replay, while Task 3 samples transitions according to their priorities and applies importance-sampling weights during optimization. After each gradient update, the TD errors of the sampled transitions are used to update their priorities in the replay buffer. New transitions are inserted with the maximum current priority so that they have a chance to be sampled soon.

2.4 Multi-step Return

In standard 1-step DQN, the target only uses the immediate reward:

$$y_t^{(1)} = r_t + \gamma(1 - d_t)Q(s_{t+1}, a')$$

To propagate delayed rewards more efficiently, we use n -step return. The accumulated reward over n steps is:

$$y_t^{(n)} = R_t^{(n)} + \gamma^n(1 - d_{t+n})Q_{target}(s_{t+n}, a^*)$$

When combined with Double DQN, the next action is selected by the online network:

$$a^* = \arg \max_{a'} Q_{online}(s_{t+n}, a')$$

and evaluated by the target network:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n(1 - d_{t+n})Q_{target}(s_{t+n}, a^*)$$

During training, the target is computed using γ^n instead of γ . In our experiments, we used $n = 3$. The n -step buffer stores consecutive transitions and computes the discounted reward before inserting the aggregated transition into the replay buffer.

2.5 Training Setup

WandB was used to track episodic reward, evaluation reward, environment steps, and epsilon decay.

Table2.

Parameter	Task1	Task2	Task3
Batch size	64	64	64
Learning rate	5e-5	5e-5	5e-5
Replay size	100k	100k	200k
Replay start	500	50k	30k
Target update	500	1000	1000
Min epsilon	0.01	0.05	0.05
Updates/step	1	1	2

3. Results & Analysis

3.1 Training Curves

Figure 1, Figure 2, and Figure 3 show the training curves for Task 1, Task 2, and Task 3, respectively. In all figures, the x-axis represents the number of environment steps, and the y-axis represents the evaluation reward.

For Task 1, the CartPole agent quickly learned a stable policy. Since the input state is low-

dimensional and the environment dynamics are relatively simple, the fully connected DQN was able to converge efficiently. The evaluation reward reached the maximum score of 500, showing that vanilla DQN is sufficient for this task.



Figure 1. Evaluation reward curve for Task 1 on CartPole-v1.

Task 2 is substantially more challenging because the agent learns directly from visual observations. With grayscale conversion, resizing, frame stacking, and a CNN-based DQN, the agent improves from negative to positive reward, although training remains noisier than CartPole.



Figure 2. Evaluation reward curve for Task 2 on Pong-v5 using vanilla DQN.

Task 3 adds Double DQN, PER, and 3-step return. The enhanced agent reaches high positive rewards relatively early, but its evaluation performance still shows substantial variance.



Figure 3. Evaluation reward curve for Task 3 on Pong-v5 using enhanced DQN.

3.2 Sample Efficiency Analysis

Compared with Task 2, Task 3 reaches positive and high rewards earlier, suggesting improved sample efficiency. However, the reward curves remain noisy. Figure 4 further shows a trade-off: the faster-learning configuration reaches high rewards sooner but fluctuates more, while the slower configuration is more stable.



Figure 4. Comparison of learning speed and stability across different Task 3 training configurations.

3.3 Limited Ablation-style Analysis

Due to limited training time, I was not able to fully retrain separate Pong agents for each individual enhancement until convergence. Therefore, I performed a limited ablation-style analysis based on the available vanilla DQN baseline from Task 2 and the full enhanced DQN from Task 3. The goal of this analysis is to explain the expected contribution of each enhancement and compare the combined enhanced agent against the vanilla baseline.

Table3. Limited Ablation-style Analysis of DQN Enhancements

Component	Main role
Vanilla DQN	Baseline with uniform replay and 1-step target
Double DQN	Reduce Q-value overestimation
PER	Prioritize transitions with large TD errors
Multi-step return	Propagate delayed rewards faster
Full Enhanced DQN	Combine all three improvements

Because the three enhancements were enabled together, the current experiment cannot isolate the individual contribution of each component.

4. Task Results

4.1 Evaluation Protocol

Models were evaluated using greedy action selection without epsilon exploration. Task 1 was evaluated over 20 CartPole episodes, while Pong models used the same preprocessing

pipeline as training. Results are reported as mean reward over multiple episodes.

4.2 Task 1: CartPole-v1

```
PS D:\NCKU\115-2\深度學習\Lab5> & C:/Users/we1/anaconda3/envs/NCKU/python.exe d:/NCKU/115-2/深度學習/Lab5/test_task1.py
Episode 0: Reward = 500.0
Episode 1: Reward = 500.0
Episode 2: Reward = 500.0
Episode 3: Reward = 500.0
Episode 4: Reward = 500.0
Episode 5: Reward = 500.0
Episode 6: Reward = 500.0
Episode 7: Reward = 500.0
Episode 8: Reward = 500.0
Episode 9: Reward = 500.0
Episode 10: Reward = 500.0
Episode 11: Reward = 500.0
Episode 12: Reward = 500.0
Episode 13: Reward = 500.0
Episode 14: Reward = 500.0
Episode 15: Reward = 500.0
Episode 16: Reward = 500.0
Episode 17: Reward = 500.0
Episode 18: Reward = 500.0
Episode 19: Reward = 500.0
=====
Rewards: [500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0, 500.0]
Average Reward over 20 episodes: 500.00
Std Reward: 0.00
Estimated Task 1 Score: 15.00 / 15
=====
```

Figure 5. Task 1 Result + terminal screenshot

4.3 Task 2: Pong-v5 Vanilla DQN

```
Saved episode 7 with total reward 21.0 -> ./eval_videos_ep1400/eval_ep7.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 8 with total reward 16.0 -> ./eval_videos_ep1400/eval_ep8.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 9 with total reward 16.0 -> ./eval_videos_ep1400/eval_ep9.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 10 with total reward 18.0 -> ./eval_videos_ep1400/eval_ep10.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 11 with total reward 21.0 -> ./eval_videos_ep1400/eval_ep11.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 12 with total reward 16.0 -> ./eval_videos_ep1400/eval_ep12.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 13 with total reward 20.0 -> ./eval_videos_ep1400/eval_ep13.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 14 with total reward 20.0 -> ./eval_videos_ep1400/eval_ep14.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 15 with total reward 18.0 -> ./eval_videos_ep1400/eval_ep15.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 16 with total reward 19.0 -> ./eval_videos_ep1400/eval_ep16.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 17 with total reward 21.0 -> ./eval_videos_ep1400/eval_ep17.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 18 with total reward 16.0 -> ./eval_videos_ep1400/eval_ep18.mp4
IMAGEIO FFMPEG WRITER WARNING: input image is not divisible by macro_block_size=16, resizing from (160, 210) to (160, 224) to ensure video com
your input image divisible by the macro_block_size or set the macro_block_size to 1 (risking incompatibility).
Saved episode 19 with total reward 18.0 -> ./eval_videos_ep1400/eval_ep19.mp4
=====
Rewards: [10.0, 13.0, 18.0, 19.0, 11.0, 20.0, 19.0, 21.0, 16.0, 16.0, 18.0, 21.0, 16.0, 20.0, 20.0, 18.0, 19.0, 21.0, 16.0, 18.0]
Average reward: 17.50
Std reward: 3.09
=====
PS D:\NCKU\115-2\深度學習\Lab5> |
```

Figure 6. Task 2 Result + terminal screenshot

4.4 Task 3: Enhanced DQN

In Task 3, I also tried multiple training configurations. One important observation is **the trade-off between learning speed and stability**. Some models learned quickly and occasionally reached high scores, but their performance varied significantly across evaluation seeds. Other models were more stable, but they improved more slowly and did not reach high scores within the limited training steps. This shows that better sample efficiency does not always lead to stable performance, especially in Atari environments such as Pong-v5.

```

=====
PS D:\NCKU\115-2\深度學習\Lab5> & C:/Users/wei/anaconda3/envs/NCKU/python.exe d:/NCKU/115-2/深度學習/Lab5/test_task3.py
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Using device: cuda
Environment: ALE/Pong-v5
Model path: ./results_task3/model_ep100.pt
=====
Evaluating seeds 0 to 19
=====
Seed 00 | Reward: 20.00 | Steps: 1768
Seed 01 | Reward: 20.00 | Steps: 1799
Seed 02 | Reward: 20.00 | Steps: 1942
Seed 03 | Reward: 18.00 | Steps: 1929
Seed 04 | Reward: 15.00 | Steps: 2435
Seed 05 | Reward: 19.00 | Steps: 1786
Seed 06 | Reward: 19.00 | Steps: 2005
Seed 07 | Reward: 18.00 | Steps: 2067
Seed 08 | Reward: 17.00 | Steps: 2122
Seed 09 | Reward: 19.00 | Steps: 1958
Seed 10 | Reward: 18.00 | Steps: 2054
Seed 11 | Reward: 20.00 | Steps: 1782
Seed 12 | Reward: 15.00 | Steps: 2019
Seed 13 | Reward: 21.00 | Steps: 1742
Seed 14 | Reward: 17.00 | Steps: 2192
Seed 15 | Reward: 18.00 | Steps: 1976
Seed 16 | Reward: 16.00 | Steps: 2216
Seed 17 | Reward: 16.00 | Steps: 1994
Seed 18 | Reward: 15.00 | Steps: 2140
Seed 19 | Reward: 19.00 | Steps: 1809
=====
Evaluation Summary
=====
Episodes: 20
Mean Reward: 18.00
Std Reward: 1.82
Min Reward: 15.00
Max Reward: 21.00
=====

```

Figure 7. Task 3 Best result + terminal screenshot

Although the enhanced DQN did not reach a 20-episode average score of 19 within 2.5M environment steps, the best checkpoint already achieved an average reward of 18.00. Therefore, the result is classified into the >2.5M category, mainly due to the limited training time.

Table4. Task 3 snapshot evaluation results over 20 episodes.

Snapshot	Mean Reward	Std Reward	Reached Avg. Score 19
600k	4.75	3.24	No
1M	14.90	4.55	No
1.5M	12.70	5.55	No
2M	15.55	6.01	No
2.5M	14.80	3.11	No

5. Conclusion

In this lab, we implemented DQN and several enhanced DQN variants, and applied them to two environments with different levels of difficulty: CartPole-v1 and Pong-v5. Through this experiment, I gained a clearer understanding of the core components of DQN, including Bellman error, experience replay, target network, and epsilon-greedy exploration.

From Task 1 on CartPole-v1, I observed that vanilla DQN is already effective for low-dimensional control tasks. Since the state space is simple and compact, a fully connected Q-network is sufficient for learning a stable policy.

From Task 2 and Task 3 on Pong-v5, I learned that reinforcement learning becomes much more challenging when the agent needs to learn from high-dimensional visual observations. The agent must first extract useful information from image frames and then learn an effective action policy through many environment interactions. This also showed me that training time, hyperparameter tuning, and evaluation stability are especially important in Atari environments.

Through Task 3, I gained a better understanding of the roles of Double DQN, Prioritized Experience Replay, and multi-step return. Double DQN helps reduce Q-value overestimation, PER allows the agent to focus more on informative transitions, and multi-step return helps propagate delayed rewards more efficiently. These methods can improve sample efficiency and help the model learn faster. However, this experiment also showed that learning faster does not always mean learning more stably. If the exploration schedule, learning rate, update frequency, or replay buffer settings are not carefully chosen, the agent may reach high scores occasionally but still show large variance across different evaluation seeds. Therefore, reinforcement learning often requires a trade-off between sample efficiency and training stability.

Although the final model did not consistently reach an average score of 19, the training curves show that the agent learned meaningful Pong strategies and was sometimes able to achieve high scores. Overall, this lab taught me that implementing a reinforcement learning algorithm is not only about coding the method itself, but also about carefully handling preprocessing, model architecture, training schedule, replay buffer design, and evaluation protocol. This experience helped me better understand the practical challenges of DQN and showed that achieving stable high performance in RL requires extensive experimentation, tuning, and rigorous evaluation.